

```

// DecodeAutonomous - Nikhil
// this one is meant to be a simplified version of what actual DECODE autonomous
// code does (DECODE is the 25/26 ftc season game). in the real game, robots have
// 30 seconds to sense the "motif" (a randomised colour pattern shown at the start
// of the match) then score artifacts (purple/green balls) into their goal, sorting
// them to match that pattern.
// i obviously can't build the real thing without the real field/game elements, but
// this mimics the actual logic flow: drive to the scoring area, read a colour sensor
// to "classify" what it's found (standing in for an artifact's colour), branch based
// on that, then return to a base zone and stop - using the same building blocks
// (encoder driving + colour/distance sensing) as my other files

public class DecodeAutonomous extends LinearOpMode {
    DcMotor motorLeft;
    DcMotor motorRight;
    ColorSensor color1;
    DistanceSensor distance1;

    @Override
    public void runOpMode() {
        motorLeft = hardwareMap.get(DcMotor.class, "motorLeft");
        motorRight = hardwareMap.get(DcMotor.class, "motorRight");
        color1 = hardwareMap.get(ColorSensor.class, "color1");
        distance1 = hardwareMap.get(DistanceSensor.class, "distance1");

        motorLeft.setDirection(DcMotor.Direction.REVERSE);
        motorLeft.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
        motorRight.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);

        waitForStart();

        // drive to the "goal" - standing in for approaching the scoring area
        moveTo(1200, 1200, 0.5);

        // read the sensor once we're there and classify what we found
        // (purple/green in the real game, using red/blue thresholds here since
        // that's what i've got set up and tested in ftcsim)
        telemetry.addData("Red Value", color1.red());
        telemetry.addData("Blue Value", color1.blue());
        telemetry.update();

        if (color1.red() > 220) {
            // "classified" one way - go to scoring position A
            moveTo(300, 300, 0.4);
        } else if (color1.blue() > 220) {
            // "classified" the other way - go to scoring position B
            moveTo(-300, -300, 0.4);
        }

        // gotta switch back out of RUN_TO_POSITION before using setPower() directly
        // again here - learned this the hard way, motors ignore setPower() properly
        // if you're still in RUN_TO_POSITION mode from the last moveTo() call
        motorLeft.setMode(DcMotor.RunMode.RUN_USING_ENCODER);
        motorRight.setMode(DcMotor.RunMode.RUN_USING_ENCODER);

        // head back to base and stop once close, same pattern as my wall
        // avoidance logic - keeps checking distance rather than guessing a time
        while (opModeIsActive() && distance1.getDistance(DistanceUnit.CM) > 30) {
            motorLeft.setPower(-0.5);
            motorRight.setPower(-0.5);
            telemetry.addData("Distance to base", distance1.getDistance(DistanceUnit.CM));
            telemetry.update();
        }
    }
}

```

```

        motorLeft.setPower(0);
        motorRight.setPower(0);
    }

    private void moveTo(int leftTicks, int rightTicks, double power) {
        int leftTarget = motorLeft.getCurrentPosition() + leftTicks;
        int rightTarget = motorRight.getCurrentPosition() + rightTicks;

        motorLeft.setTargetPosition(leftTarget);
        motorRight.setTargetPosition(rightTarget);

        motorLeft.setMode(DcMotor.RunMode.RUN_TO_POSITION);
        motorRight.setMode(DcMotor.RunMode.RUN_TO_POSITION);

        motorLeft.setPower(power);
        motorRight.setPower(power);

        while (opModeIsActive() && motorLeft.isBusy() && motorRight.isBusy()) {
            telemetry.addData("Left ticks", motorLeft.getCurrentPosition());
            telemetry.addData("Right ticks", motorRight.getCurrentPosition());
            telemetry.update();
        }

        motorLeft.setPower(0);
        motorRight.setPower(0);
    }
}

```